

---

# KHẢO SÁT TOÀN DIỆN SỰ PHÁT TRIỂN CỦA YOLO

*Từ YOLOv1 (2015) đến YOLO26 (2026)*

Kiến trúc – Phương pháp – Kết quả – Xu hướng

---

**Nguyễn Cao Trị**

Ngày biên soạn: Ngày 14 tháng 5 năm 2026

# Mục lục

|          |                                                               |          |
|----------|---------------------------------------------------------------|----------|
| <b>1</b> | <b>MỞ ĐẦU</b>                                                 | <b>5</b> |
| 1.1      | Giới thiệu bài toán Object Detection . . . . .                | 5        |
| 1.2      | YOLO — Bước ngoặt . . . . .                                   | 5        |
| 1.3      | Mục tiêu và phạm vi báo cáo . . . . .                         | 6        |
| <b>2</b> | <b>NỀN TẢNG LÝ THUYẾT</b>                                     | <b>7</b> |
| 2.1      | Pipeline Object Detection hiện đại . . . . .                  | 7        |
| 2.2      | One-stage vs Two-stage Detectors . . . . .                    | 7        |
| 2.3      | Các khái niệm cốt lõi . . . . .                               | 7        |
| 2.3.1    | IoU (Intersection over Union) . . . . .                       | 7        |
| 2.3.2    | mAP (mean Average Precision) . . . . .                        | 8        |
| 2.3.3    | Anchor Boxes . . . . .                                        | 8        |
| 2.3.4    | NMS (Non-Maximum Suppression) . . . . .                       | 8        |
| 2.3.5    | Các loại Loss Function trong YOLO . . . . .                   | 8        |
| 2.4      | Datasets chuẩn . . . . .                                      | 8        |
| <b>3</b> | <b>THẾ HỆ ĐẦU: YOLOv1 – YOLOv4 (2015–2020)</b>                | <b>9</b> |
| 3.1      | YOLOv1 (2015) — Khởi nguồn . . . . .                          | 9        |
| 3.1.1    | Ý tưởng cốt lõi . . . . .                                     | 9        |
| 3.1.2    | Kiến trúc . . . . .                                           | 9        |
| 3.1.3    | Hàm Loss (5 thành phần) . . . . .                             | 9        |
| 3.1.4    | Kết quả và đánh giá . . . . .                                 | 10       |
| 3.2      | YOLOv2 / YOLO9000 (2016) — Better, Faster, Stronger . . . . . | 10       |
| 3.2.1    | Các cải tiến chính . . . . .                                  | 10       |
| 3.2.2    | Darknet-19 & YOLO9000 . . . . .                               | 11       |
| 3.3      | YOLOv3 (2018) — Multi-Scale Detection . . . . .               | 11       |
| 3.3.1    | Đột phá: 3 scales . . . . .                                   | 11       |
| 3.3.2    | Darknet-53 . . . . .                                          | 11       |
| 3.3.3    | Kết quả . . . . .                                             | 11       |
| 3.4      | YOLOv4 (2020) — Tập đại thành . . . . .                       | 11       |
| 3.4.1    | Phương pháp luận: BoF & BoS . . . . .                         | 11       |
| 3.4.2    | Kiến trúc: CSPDarknet53 + SPP + PAN . . . . .                 | 12       |

|          |                                                                 |           |
|----------|-----------------------------------------------------------------|-----------|
| 3.4.3    | Kỹ thuật mới . . . . .                                          | 12        |
| 3.4.4    | Kết quả . . . . .                                               | 12        |
| <b>4</b> | <b>THỂ HỆ GIỮA: YOLOv5 – YOLOv8 (2020–2023)</b>                 | <b>13</b> |
| 4.1      | YOLOv5 (2020) — Dân chủ hóa YOLO . . . . .                      | 13        |
| 4.1.1    | Ý nghĩa lịch sử . . . . .                                       | 13        |
| 4.1.2    | Kiến trúc . . . . .                                             | 13        |
| 4.1.3    | 5 model sizes . . . . .                                         | 14        |
| 4.2      | YOLOv6 (2022) — Industrial Deployment . . . . .                 | 14        |
| 4.2.1    | Bối cảnh . . . . .                                              | 14        |
| 4.2.2    | Re-parameterization — Kỹ thuật chủ đạo . . . . .                | 14        |
| 4.2.3    | Kiến trúc chi tiết . . . . .                                    | 15        |
| 4.2.4    | Quantization Pipeline — Điểm mạnh nhất . . . . .                | 15        |
| 4.2.5    | Kết quả . . . . .                                               | 15        |
| 4.3      | YOLOv7 (2022) — SOTA Accuracy . . . . .                         | 16        |
| 4.3.1    | E-ELAN (Extended Efficient Layer Aggregation Network) . . . . . | 16        |
| 4.3.2    | Planned Re-parameterized Conv . . . . .                         | 16        |
| 4.3.3    | Coarse-to-Fine Label Assignment . . . . .                       | 16        |
| 4.3.4    | Kết quả chi tiết . . . . .                                      | 17        |
| 4.4      | YOLOv8 (2023) — Đa năng nhất . . . . .                          | 17        |
| 4.4.1    | 3 thay đổi kiến trúc lớn so với v5 . . . . .                    | 17        |
| 4.4.2    | C2f Module . . . . .                                            | 18        |
| 4.4.3    | Multi-task — 6 tác vụ trong 1 framework . . . . .               | 18        |
| 4.4.4    | Kết quả . . . . .                                               | 18        |
| <b>5</b> | <b>THỂ HỆ MỚI: YOLOv9 – YOLOv12 (2024–2025)</b>                 | <b>19</b> |
| 5.1      | YOLOv9 (2024) — Chống mất thông tin . . . . .                   | 19        |
| 5.1.1    | Vấn đề: Information Bottleneck . . . . .                        | 19        |
| 5.1.2    | PGI (Programmable Gradient Information) . . . . .               | 19        |
| 5.1.3    | GELAN (Generalized ELAN) . . . . .                              | 20        |
| 5.1.4    | Kết quả chi tiết . . . . .                                      | 20        |
| 5.2      | YOLOv10 (2024) — NMS-Free . . . . .                             | 21        |
| 5.2.1    | Vấn đề NMS . . . . .                                            | 21        |
| 5.2.2    | Consistent Dual Assignments . . . . .                           | 21        |
| 5.2.3    | Efficiency-Driven Design . . . . .                              | 22        |
| 5.2.4    | Kết quả . . . . .                                               | 22        |
| 5.3      | YOLO11 (2024) — Nâng cấp toàn diện . . . . .                    | 22        |
| 5.3.1    | C3K2 (CSP Bottleneck with 2 Kernel sizes) . . . . .             | 22        |
| 5.3.2    | C2PSA (Cross Stage Partial Spatial Attention) . . . . .         | 23        |

|          |                                                      |           |
|----------|------------------------------------------------------|-----------|
| 5.3.3    | Kết quả . . . . .                                    | 23        |
| 5.4      | YOLOv12 (2025) — Kỷ nguyên Attention . . . . .       | 23        |
| 5.4.1    | Thách thức đưa Attention vào YOLO . . . . .          | 23        |
| 5.4.2    | Area Attention ( $A^2$ ) . . . . .                   | 24        |
| 5.4.3    | R-ELAN (Residual ELAN) . . . . .                     | 24        |
| 5.4.4    | Tối ưu phần cứng . . . . .                           | 24        |
| 5.4.5    | Kết quả chi tiết . . . . .                           | 25        |
| <b>6</b> | <b>HIỆN TẠI VÀ TƯƠNG LAI: YOLO26 (2026)</b>          | <b>26</b> |
| 6.1      | Tại sao gọi “YOLO26” thay vì “YOLOv13”? . . . . .    | 26        |
| 6.2      | Bỏ DFL — Thay đổi triết lý lớn nhất . . . . .        | 26        |
| 6.2.1    | Phân tích kỹ thuật: Tại sao bỏ DFL? . . . . .        | 26        |
| 6.3      | MuSGD (Muon-enhanced SGD) . . . . .                  | 27        |
| 6.4      | STAL (Small-Target-Aware Label Assignment) . . . . . | 28        |
| 6.5      | ProgLoss (Progressive Loss Balancing) . . . . .      | 28        |
| 6.6      | Kết quả . . . . .                                    | 29        |
| 6.7      | Tổng kết triết lý YOLO26 . . . . .                   | 29        |
| <b>7</b> | <b>BƯỚC NGOẶT OPEN-VOCABULARY</b>                    | <b>31</b> |
| 7.1      | YOLO-World (2024) — Tiên phong . . . . .             | 31        |
| 7.1.1    | Ý tưởng . . . . .                                    | 31        |
| 7.1.2    | Kết quả . . . . .                                    | 31        |
| 7.2      | YOLOE (2025) — Real-Time Seeing Anything . . . . .   | 31        |
| 7.2.1    | Vấn đề . . . . .                                     | 31        |
| 7.2.2    | Ba chế độ Prompt . . . . .                           | 32        |
| 7.2.3    | Kết quả . . . . .                                    | 32        |
| 7.2.4    | Ý nghĩa . . . . .                                    | 32        |
| <b>8</b> | <b>CÁC BIẾN THỂ YOLO</b>                             | <b>33</b> |
| 8.1      | YOLOX (2021) — Anchor-Free tiên phong . . . . .      | 33        |
| 8.2      | PP-YOLOE (2022) — Baidu Industrial . . . . .         | 33        |
| 8.3      | YOLO-NAS (2023) — AI thiết kế AI . . . . .           | 33        |
| 8.4      | Bảng so sánh biến thể . . . . .                      | 34        |
| <b>9</b> | <b>SO SÁNH TỔNG HỢP, XU HƯỚNG VÀ KHUYẾN NGHỊ</b>     | <b>35</b> |
| 9.1      | Bảng tổng quan tất cả phiên bản . . . . .            | 35        |
| 9.2      | Tiến hóa kiến trúc . . . . .                         | 36        |
| 9.3      | Biểu đồ Pareto: Params vs mAP . . . . .              | 37        |
| 9.4      | Trade-off: Version “thua” đời trước . . . . .        | 37        |
| 9.5      | Xếp hạng theo tiêu chí . . . . .                     | 38        |

|           |                                               |           |
|-----------|-----------------------------------------------|-----------|
| 9.5.1     | Accuracy thuần túy (mAP COCO@50:95) . . . . . | 38        |
| 9.5.2     | Efficiency (mAP/Params) . . . . .             | 38        |
| 9.5.3     | Production readiness (2026) . . . . .         | 38        |
| 9.6       | Sơ đồ phả hệ YOLO (Family Tree) . . . . .     | 39        |
| 9.7       | Xu hướng phát triển . . . . .                 | 39        |
| 9.8       | Khuyến nghị chọn model . . . . .              | 40        |
| <b>10</b> | <b>KẾT LUẬN</b>                               | <b>41</b> |
|           | <b>Phụ lục: Bảng thuật ngữ</b>                | <b>43</b> |

# CHƯƠNG 1

## MỞ ĐẦU

### 1.1 Giới thiệu bài toán Object Detection

---

Phát hiện vật thể (Object Detection) là một trong những bài toán trung tâm của thị giác máy tính (Computer Vision). Mục tiêu là xác định **vị trí** (bounding box) và **loại** (class) của tất cả vật thể trong ảnh hoặc video. Object Detection được ứng dụng rộng rãi trong xe tự hành, giám sát an ninh, y tế, công nghiệp, nông nghiệp và nhiều lĩnh vực khác.

Trước khi YOLO ra đời (2015), các phương pháp detection chủ đạo là **two-stage detectors**:

- **R-CNN** (2014): Dùng Selective Search tạo 2000 region proposals, mỗi region qua CNN trích features, rồi SVM phân loại. Cực chậm: >40 giây/ảnh.
- **Fast R-CNN** (2015): Cải tiến bằng cách chia sẻ CNN features, nhưng vẫn cần Selective Search.
- **Faster R-CNN** (2015): Thay Selective Search bằng Region Proposal Network (RPN), nhưng vẫn là pipeline 2 giai đoạn.

Điểm chung của các phương pháp trên: pipeline **rời rạc**, mỗi thành phần tối ưu riêng, khó đạt real-time.

### 1.2 YOLO — Bước ngoặt

---

YOLO (You Only Look Once) ra đời năm 2015, thay đổi hoàn toàn cách tiếp cận: coi object detection là **bài toán hồi quy duy nhất**, từ pixel ảnh trực tiếp đến bounding boxes và class probabilities. Chỉ cần **một lần forward pass** qua mạng neural → real-time (45 FPS).

Từ 2015 đến 2026, YOLO đã trải qua **14 phiên bản chính** (v1–v12, YOLOE, YOLO26) và nhiều biến thể (YOLOX, PP-YOLOE, YOLO-NAS, YOLO-World), với mAP tăng từ 63.4% (VOC) lên 57.2% (COCO), tốc độ từ 45 FPS lên hàng trăm FPS.

---

## 1.3 Mục tiêu và phạm vi báo cáo

---

**Mục tiêu:** Khảo sát toàn diện sự phát triển của YOLO qua các đời, phân tích kiến trúc, phương pháp, kết quả, ưu/nhược điểm, và xu hướng tương lai.

**Phạm vi:**

- Dòng chính: YOLOv1  $\rightarrow$  YOLOv12, YOLOE, YOLO26
- Biến thể: YOLOX, PP-YOLOE, YOLO-NAS, YOLO-World
- So sánh đa chiều và khuyến nghị lựa chọn

## CHƯƠNG 2

# NỀN TẢNG LÝ THUYẾT

## 2.1 Pipeline Object Detection hiện đại

---

Một detector hiện đại gồm 3 phần:

- **Backbone:** Mạng trích xuất features từ ảnh (VGG, ResNet, Darknet, CSPDarknet, EfficientNet...)
- **Neck:** Kết hợp features ở nhiều scales (FPN, PAN, BiFPN)
- **Head:** Dự đoán bounding boxes và classes

## 2.2 One-stage vs Two-stage Detectors

---

**Bảng 2.1:** So sánh One-stage và Two-stage

| Tiêu chí | Two-stage            | One-stage          |
|----------|----------------------|--------------------|
| Dải diện | Faster R-CNN         | YOLO, SSD          |
| Tốc độ   | Chậm (5–20 FPS)      | Nhanh (30–300 FPS) |
| Accuracy | Cao hơn (trước 2020) | Bắt kịp từ YOLOv4+ |
| Pipeline | 2 giai đoạn          | End-to-end         |

## 2.3 Các khái niệm cốt lõi

---

### 2.3.1 IoU (Intersection over Union)

Đo mức trùng lặp giữa predicted box và ground truth:

$$\text{IoU} = \frac{|B_p \cap B_{gt}|}{|B_p \cup B_{gt}|}$$



### 2.3.2 mAP (mean Average Precision)

- **mAP@50:** Tính AP ở ngưỡng IoU = 0.5
- **mAP@50:95:** Trung bình AP ở IoU từ 0.5 đến 0.95 (bước 0.05) — metric chính trên COCO

### 2.3.3 Anchor Boxes

Các hộp mẫu (templates) được định nghĩa trước với kích thước và tỷ lệ khác nhau. Model dự đoán **offsets** so với anchors thay vì tọa độ tuyệt đối.

### 2.3.4 NMS (Non-Maximum Suppression)

Thuật toán loại bỏ bounding boxes trùng lặp: giữ box có confidence cao nhất, loại bỏ các boxes có IoU > ngưỡng với box đã chọn.

### 2.3.5 Các loại Loss Function trong YOLO

**Bảng 2.2:** Tiến hóa Loss Function

| Loss     | Version | Đặc điểm                              |
|----------|---------|---------------------------------------|
| MSE/SSE  | v1–v3   | Đơn giản, coi box lớn/nhỏ ngang nhau  |
| IoU Loss | –       | Trực tiếp tối ưu IoU                  |
| GIoU     | v6      | Xét cả vùng bao ngoài                 |
| CIoU     | v4–v8   | + khoảng cách tâm + aspect ratio      |
| DFL      | v8–v12  | Dự đoán phân phối thay vì giá trị đơn |

## 2.4 Datasets chuẩn

- **PASCAL VOC** (2007/2012): 20 classes, 10K ảnh. Metric: mAP@50.
- **MS COCO:** 80 classes, 330K ảnh. Metric: mAP@50:95. Benchmark chính từ v3.
- **LVIS:** 1203 categories. Dùng cho open-vocabulary (YOLOE).

## CHƯƠNG 3

# THỂ HỆ ĐẦU: YOLO<sub>v1</sub> – YOLO<sub>v4</sub> (2015–2020)

### 3.1 YOLO<sub>v1</sub> (2015) — Khởi nguồn

---

**Paper:** “You Only Look Once: Unified, Real-Time Object Detection” (CVPR 2016) [1]

**Tác giả:** Joseph Redmon, Santosh Divvala, Ross Girshick, Ali Farhadi

[\* YÊU CẦU CHÈN ẢNH: Tác giả hãy chèn sơ đồ kiến trúc YOLO<sub>v1</sub> từ paper gốc (Figure 3, Redmon et al. 2016).]

#### 3.1.1 Ý tưởng cốt lõi

Coi object detection là **bài toán hồi quy duy nhất**: từ pixel ảnh  $\rightarrow$  bounding boxes + class probabilities, chỉ qua **một lần forward pass**. Chia ảnh thành lưới  $S \times S$  ( $S = 7$ ), mỗi ô dự đoán  $B = 2$  boxes và  $C = 20$  classes [1].

#### 3.1.2 Kiến trúc

24 lớp convolutional + 2 lớp fully connected. Cảm hứng từ GoogLeNet. Output: tensor  $7 \times 7 \times 30$ . Fast YOLO: 9 conv layers.

#### 3.1.3 Hàm Loss (5 thành phần)

$$\mathcal{L} = \lambda_{coord} \sum_i \sum_j \mathbb{K}_{ij}^{obj} \left[ (x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 + (\sqrt{w_i} - \sqrt{\hat{w}_i})^2 + (\sqrt{h_i} - \sqrt{\hat{h}_i})^2 \right] \quad (3.1)$$

$$+ \sum_i \sum_j \mathbb{K}_{ij}^{obj} (C_i - \hat{C}_i)^2 + \lambda_{noobj} \sum_i \sum_j \mathbb{K}_{ij}^{noobj} (C_i - \hat{C}_i)^2 + \sum_i \mathbb{K}_i^{obj} \sum_c (p_i(c) - \hat{p}_i(c))^2 \quad (3.2)$$

Với  $\lambda_{coord} = 5$ ,  $\lambda_{noobj} = 0.5$ . Dùng  $\sqrt{w}$ ,  $\sqrt{h}$  để giảm ảnh hưởng sai số ở box lớn [1].

### 3.1.4 Kết quả và đánh giá

**Bảng 3.1:** YOLOv1 trên PASCAL VOC 2007

| Model       | mAP   | FPS |
|-------------|-------|-----|
| Fast YOLO   | 52.7% | 155 |
| YOLO        | 63.4% | 45  |
| YOLO VGG-16 | 66.4% | 21  |
| Fast R-CNN  | 70.0% | 0.5 |

*Ghi chú: FPS đo trên Titan X (Maxwell). Không so sánh trực tiếp với FPS đo trên các thế hệ GPU sau (V100, T4...).*

**Ưu điểm:** Real-time đầu tiên; ít false positive background; tổng quát hóa tốt (Picasso: 53.3% vs R-CNN 10.2%) [1].

**Nhược điểm:** Vật nhỏ kém (grid thô  $7 \times 7$ ); localization sai nhiều; mỗi ô chỉ 1 class.

## 3.2 YOLOv2 / YOLO9000 (2016) — Better, Faster, Stronger

**Paper:** “YOLO9000: Better, Faster, Stronger” (CVPR 2017) [2]

**Tác giả:** Joseph Redmon, Ali Farhadi

### 3.2.1 Các cải tiến chính

**Bảng 3.2:** Cải tiến tích lũy YOLOv2

| #                   | Kỹ thuật                                                  | mAP (%)     | Thay đổi     |
|---------------------|-----------------------------------------------------------|-------------|--------------|
| 0                   | Baseline (v1)                                             | 63.4        | –            |
| 1                   | + Batch Normalization                                     | 65.4        | +2.0         |
| 2                   | + High-Res Classifier (448)                               | 69.4        | +4.0         |
| 3                   | + Anchor Boxes (5 anchors)                                | 69.2        | Recall +7%   |
| 4                   | + Dimension Clusters (k-means)                            | –           | IoU tốt hơn  |
| 5                   | + Direct Location (sigmoid)                               | 74.4        | +5.0         |
| 6                   | + Passthrough ( $26 \times 26 \rightarrow 13 \times 13$ ) | 75.4        | +1.0         |
| 7                   | + Multi-Scale Training                                    | 76.8        | +1.4         |
| <b>YOLOv2 (416)</b> |                                                           | <b>76.8</b> | <b>+13.4</b> |

### 3.2.2 Darknet-19 & YOLO9000

Backbone mới: 19 conv + 5 maxpool, 5.58B FLOPs (VGG: 30.69B). YOLO9000 dùng **WordTree** joint training trên COCO + ImageNet → detect 9000+ categories [2].

## 3.3 YOLOv3 (2018) — Multi-Scale Detection

**Paper:** “YOLOv3: An Incremental Improvement” (Tech Report) [3]

**Tác giả:** Joseph Redmon, Ali Farhadi (*paper cuối cùng của Redmon trước khi dừng nghiên cứu CV vì lo ngại đạo đức*)

### 3.3.1 Đột phá: 3 scales

Dự đoán ở 3 scales ( $13 \times 13$ ,  $26 \times 26$ ,  $52 \times 52$ ) theo FPN-style, 3 anchors/scale = 9 anchors tổng. Thay Softmax bằng **Sigmoid** (multi-label classification) [3].

### 3.3.2 Darknet-53

53 conv layers + residual connections. Chính xác bằng ResNet-152 nhưng **nhANH gấp 2×** (78 vs 37 FPS) [3].

### 3.3.3 Kết quả

mAP50: 57.9% (COCO), ngang RetinaNet nhưng nhanh gấp 3.8×. mAP50-95: 33.0% — cho thấy localization vẫn yếu ở IoU cao.

## 3.4 YOLOv4 (2020) — Tập đại thành

**Paper:** “YOLOv4: Optimal Speed and Accuracy of Object Detection” [4]

**Tác giả:** Alexey Bochkovskiy, Chien-Yao Wang, Hong-Yuan Mark Liao

[\* YÊU CẦU CHÈN ẢNH: Tác giả hãy chèn sơ đồ kiến trúc YOLOv4 (CSPDarknet53 + SPP + PAN) từ paper gốc.]

### 3.4.1 Phương pháp luận: BoF & BoS

- **Bag of Freebies (BoF):** Chỉ tăng training cost — Mosaic, CutMix, CIoU Loss, Label Smoothing, SAT, DropBlock, Cosine Annealing

- **Bag of Specials (BoS):** Tăng nhẹ inference — Mish activation, SPP, SAM, PAN, DIoU-NMS

### 3.4.2 Kiến trúc: CSPDarknet53 + SPP + PAN

CSP (Cross Stage Partial) chia features thành 2 phần, giảm 20% computation. SPP tăng receptive field. PAN cải thiện localization signals [4].

### 3.4.3 Kỹ thuật mới

- **Mosaic Augmentation:** Ghép 4 ảnh  $\rightarrow$  model thấy nhiều context, giảm batch size cần thiết
- **SAT (Self-Adversarial Training):** Tự tạo adversarial perturbation  $\rightarrow$  train robust hơn
- **CIoU Loss:**  $\mathcal{L}_{CIoU} = 1 - IoU + \frac{\rho^2(b, b^{gt})}{c^2} + \alpha v$  — xét overlap + khoảng cách tâm + aspect ratio [4]

### 3.4.4 Kết quả

43.5% mAP (COCO),  $\sim 65$  FPS (V100) [4]. Nhảy vọt **+10.5%** so với v3 — bước nhảy lớn nhất lịch sử YOLO. Train được trên single GPU.

*Ghi chú: FPS đo trên V100 GPU. Không so sánh trực tiếp với FPS đo trên Titan X (v1–v3) do sự khác biệt thế hệ phần cứng.*

## CHƯƠNG 4

# THẾ HỆ GIỮA: YOLOv5 – YOLOv8 (2020–2023)

## 4.1 YOLOv5 (2020) — Dân chủ hóa YOLO

---

**Tác giả:** Glenn Jocher / Ultralytics

**Framework:** PyTorch

*Không có paper*

### 4.1.1 Ý nghĩa lịch sử

Chuyển từ Darknet (C) sang PyTorch. Release chỉ 2 tháng sau v4, gây tranh cãi vì không có paper. Tuy nhiên trở thành **YOLO phổ biến nhất** nhờ `pip install ultralytics`.

*Ví dụ:* Trước v5, muốn dùng YOLO phải compile code C, cấu hình file `.cfg` phức tạp. v5 đơn giản hóa thành 3 dòng Python:

```
from ultralytics import YOLO
model = YOLO('yolov5s.pt')
results = model('anh.jpg')
```

### 4.1.2 Kiến trúc

- **Backbone:** CSPDarknet53 (modified), **C3 module** — CSP Bottleneck 3 convolutions. C3 chia features thành 2 nhánh: 1 nhánh qua bottleneck, 1 nhánh bypass → concat.
- **Neck:** PANet + **SPPF** (SPP Fast) — thay 3 MaxPool song song bằng 3 MaxPool **nối tiếp**. Kết quả tương đương SPP nhưng nhanh hơn  $\sim 2\times$ .
- **Head:** Anchor-based, 3 scales. **AutoAnchor:** dùng k-means + genetic algorithm tự tìm anchors tối ưu cho dataset riêng.

*Ví dụ SPPF:* SPP dùng 3 MaxPool kích thước  $5 \times 5$ ,  $9 \times 9$ ,  $13 \times 13$  chạy **cùng lúc** (3 operations). SPPF dùng 3 MaxPool  $5 \times 5$  chạy **nối tiếp** — output tương đương nhưng chỉ cần 1 loại kernel, nhanh hơn trên GPU.

### 4.1.3 5 model sizes

**Bảng 4.1:** YOLOv5 model variants (COCO val2017, input 640)

| Model | Params | FLOPs  | mAP50-95 | Phù hợp      |
|-------|--------|--------|----------|--------------|
| v5n   | 1.9M   | 4.5G   | 28.0%    | Mobile, IoT  |
| v5s   | 7.2M   | 16.5G  | 37.4%    | Edge devices |
| v5m   | 21.2M  | 49.0G  | 45.4%    | Cân bằng     |
| v5l   | 46.5M  | 109.1G | 49.0%    | Server       |
| v5x   | 86.7M  | 205.7G | 50.7%    | Max accuracy |

**Đóng góp chính:** SPPF, AutoAnchor, 5 sizes, export 10+ formats, hyperparameter evolution.

**Hạn chế:** Không có paper nên không peer-reviewed; kiến trúc gần giống v4; vẫn anchor-based.

## 4.2 YOLOv6 (2022) — Industrial Deployment

**Paper:** “YOLOv6: A Single-Stage Object Detection Framework for Industrial Applications”

**Tác giả:** Meituan Vision AI (dịch vụ giao đồ ăn lớn nhất Trung Quốc)

### 4.2.1 Bối cảnh

Meituan cần detect vật thể real-time trên hàng triệu đơn hàng mỗi ngày. Yêu cầu: **cực nhanh** trên GPU server + khả năng quantize INT8 mà không mất accuracy.

### 4.2.2 Re-parameterization — Kỹ thuật chủ đạo

Kiến trúc khác nhau giữa training và inference:

*Ảnh dụ cho người không chuyên*

**Ôn thi:** Khi ôn (training), bạn đọc 5 cuốn sách, ghi chú, làm bài tập (multi-branch). Khi vào phòng thi (inference), bạn chỉ cần 1 tờ tóm tắt (single branch) chứa tinh hoa từ 5 cuốn.

- **Training:** Multi-branch — Conv  $3 \times 3$  + Conv  $1 \times 1$  + Identity song song  $\rightarrow$  học phong phú
- **Inference:** Gộp (re-parameterize) thành 1 Conv  $3 \times 3$  duy nhất  $\rightarrow$  nhanh
- **Toán học:**  $W_{merged} = W_{3 \times 3} + \text{pad}(W_{1 \times 1}) + I$  — gộp weights bằng phép cộng ma trận

### 4.2.3 Kiến trúc chi tiết

- **Backbone — EfficientRep:** Thay CSPDarknet bằng RepVGG blocks. Training: multi-branch. Inference: single-path Conv.
- **Neck — Rep-PAN:** PAN cũng dùng RepVGG blocks → inference nhanh hơn.
- **Head — Efficient Decoupled Head:** Tách cls/reg (giống YOLOX) nhưng giảm 1 Conv layer → tiết kiệm computation.
- **Label Assignment:** SimOTA (giai đoạn 1) → **TAL** (giai đoạn 2): Task-Aligned Learning căn chỉnh cls score và localization quality.
- **Loss:** VFL (Varifocal Loss) + SIoU/GIoU + DFL (model lớn).
- **Anchor-free:** Dự đoán trực tiếp 4 distances từ tâm (FCOS-style).

### 4.2.4 Quantization Pipeline — Điểm mạnh nhất

1. **PTQ** (Post-Training Quantization): Quantize model đã train → nhanh nhưng mất accuracy.
2. **QAT** (Quantization-Aware Training): Train lại với simulated quantization → giữ accuracy.
3. **RepOptimizer:** Tối ưu re-param blocks cho quantization.

Kết quả: INT8 chỉ mất **0.2% mAP** nhưng tăng **75% FPS**.

### 4.2.5 Kết quả

**Bảng 4.2:** YOLOv6 so sánh (COCO val, T4 GPU TensorRT FP16)

| Model           | mAP50-95     | FPS         | Params       | So sánh             |
|-----------------|--------------|-------------|--------------|---------------------|
| YOLOv5-N        | 28.0%        | 602         | 1.9M         | —                   |
| <b>YOLOv6-N</b> | <b>37.5%</b> | <b>1187</b> | <b>4.7M</b>  | +9.5%, 2× nhanh hơn |
| YOLOv5-S        | 37.4%        | 349         | 7.2M         | —                   |
| <b>YOLOv6-S</b> | <b>45.0%</b> | <b>495</b>  | <b>18.5M</b> | +7.6%               |
| YOLOv5-L        | 49.0%        | 113         | 46.5M        | —                   |
| <b>YOLOv6-L</b> | <b>52.8%</b> | <b>116</b>  | <b>59.6M</b> | +3.8%               |

**Nhận xét:** v6 vượt v5 ở **mọi scale**, đặc biệt mạnh ở Nano (1187 FPS!). Tuy nhiên ecosystem nhỏ hơn v5 nhiều.

## 4.3 YOLOv7 (2022) — SOTA Accuracy



**Paper:** “YOLOv7: Trainable Bag-of-Freebies Sets New State-of-the-Art for Real-Time Object Detectors” (CVPR 2023)

**Tác giả:** Chien-Yao Wang, Alexey Bochkovskiy, Hong-Yuan Mark Liao

### 4.3.1 E-ELAN (Extended Efficient Layer Aggregation Network)

Mở rộng ELAN bằng cách tăng **cardinality** (số nhánh song song) mà **không phá gradient path gốc**.

*Ấn dụ cho người không chuyên*

**Dây chuyền sản xuất:** ELAN giống 1 dây chuyền với 3 công nhân nối tiếp. E-ELAN thêm “ca kíp” mới (expand cardinality) nhưng giữ nguyên dây chuyền cũ, rồi **shuffle + merge** kết quả → output phong phú hơn.

- **Expand:** Tăng số computational blocks ( $4 \rightarrow 8$  branches)
- **Shuffle:** Xáo trộn channels giữa các branches
- **Merge:** Gộp features từ tất cả branches
- Kết quả: +0.7% mAP so với ELAN chuẩn

### 4.3.2 Planned Re-parameterized Conv

Phát hiện quan trọng: RepConv (từ RepVGG) **không nên dùng** khi có identity connection (residual). Kết hợp identity + conv  $1 \times 1$  + conv  $3 \times 3$  gây **xung đột gradient**.

**Giải pháp — RepConvN:** Bỏ identity branch, chỉ giữ conv  $1 \times 1$  + conv  $3 \times 3$ . Tăng 0.5% AP với cùng speed.

### 4.3.3 Coarse-to-Fine Label Assignment

- **Lead Head** (fine-grained): Head chính, gán nhãn chặt chẽ
- **Auxiliary Head** (coarse-grained): Head phụ, gán nhãn lỏng hơn → nhiều positive samples hơn → train tốt hơn
- Auxiliary head **chỉ dùng khi training**, bỏ khi inference → zero inference cost

*Ấn dụ cho người không chuyên*

**Giáo viên và trợ giảng:** Lead head giống giáo viên chấm thi nghiêm khắc (chỉ cho điểm khi đúng hoàn toàn). Auxiliary head giống trợ giảng nhẹ nhàng (cho điểm khi gần đúng) → sinh viên nhận nhiều feedback → học tốt hơn.

### 4.3.4 Kết quả chi tiết

**Bảng 4.3:** YOLOv7 Base models (V100 GPU)

| Model           | Params       | FPS        | mAP50-95     | So sánh               |
|-----------------|--------------|------------|--------------|-----------------------|
| YOLOv5-L        | 46.5M        | 99         | 48.8%        | —                     |
| YOLOX-L         | 54.2M        | 68         | 50.0%        | —                     |
| PPYOLOE-L       | 52.2M        | 78         | 51.4%        | —                     |
| <b>YOLOv7</b>   | <b>36.9M</b> | <b>161</b> | <b>51.4%</b> | Ít params, nhanh nhất |
| <b>YOLOv7-X</b> | <b>71.3M</b> | <b>114</b> | <b>53.1%</b> | Vượt PPYOLOE          |

**Bảng 4.4:** YOLOv7 Large models — SOTA

| Model                     | FPS       | mAP50-95     |
|---------------------------|-----------|--------------|
| YOLOv7-W6                 | 84        | 54.9%        |
| YOLOv7-E6                 | 56        | 55.9%        |
| <b>YOLOv7-E6E</b>         | <b>36</b> | <b>56.8%</b> |
| SWIN-L Cascade R-CNN      | 12        | 53.9%        |
| ConvNeXt-XL Cascade R-CNN | —         | 55.2%        |

**Kết luận:** YOLOv7-E6E (56.8%) vượt tất cả — cả Transformer-based (SWIN-L). SOTA accuracy cho đến khi YOLO26 ra đời.

## 4.4 YOLOv8 (2023) — Đa năng nhất

Tác giả: Ultralytics

Không có paper

### 4.4.1 3 thay đổi kiến trúc lớn so với v5

- Anchor-Free:** Bỏ hoàn toàn anchor boxes. Dự đoán **center point + 4 distances** (khoảng cách từ tâm đến 4 cạnh box).  
*Ví dụ:* Anchor-based giống chọn khung ảnh có sẵn (S/M/L) rồi chỉnh. Anchor-free giống vẽ tự do — đứng tại tâm vật, đo 4 hướng lên/xuống/trái/phải.
- Decoupled Head:** Tách classification và regression thành 2 branches hoàn toàn riêng biệt.  
*Ví dụ:* Thay vì 1 bác sĩ vừa chẩn đoán bệnh vừa chỉ vị trí đau, chia thành 2 bác sĩ chuyên môn — mỗi người làm tốt hơn.

3. **DFL (Distribution Focal Loss)**: Thay vì dự đoán 1 giá trị cho mỗi tọa độ box, dự đoán **phân phối xác suất** trên 16 bins.

*Ví dụ*: Thay vì nói “cạnh trái cách tâm 25px” (1 số), DFL nói “xác suất 10% ở 24px, 70% ở 25px, 20% ở 26px” → phản ánh **mức độ không chắc chắn**, localization chính xác hơn.

#### 4.4.2 C2f Module

Kế thừa C3 (v5) + cảm hứng từ ELAN (v7):

- C3: chỉ dùng output cuối cùng của bottleneck chain
- **C2f: concat output của TẤT CẢ bottlenecks** → gradient flow phong phú hơn, nhiều scale features hơn

#### 4.4.3 Multi-task — 6 tác vụ trong 1 framework

**Bảng 4.5:** 6 tác vụ YOLOv8 hỗ trợ

| Tác vụ          | Output              | Ví dụ ứng dụng          |
|-----------------|---------------------|-------------------------|
| Detection       | Bounding boxes      | Phát hiện xe, người     |
| Segmentation    | Pixel-level masks   | Tách nền ảnh            |
| Classification  | Class label         | Phân loại sản phẩm      |
| Pose Estimation | Keypoints (17 điểm) | Phân tích tư thế        |
| OBB             | Rotated boxes       | Ảnh vệ tinh, tàu thuyền |
| Tracking        | Box + Track ID      | Đếm người, theo dõi xe  |

#### 4.4.4 Kết quả

**Bảng 4.6:** YOLOv8 so sánh (COCO val2017, input 640)

| Model | Params | FLOPs  | mAP50-95 | So với v5          |
|-------|--------|--------|----------|--------------------|
| v8n   | 3.2M   | 8.7G   | 37.3%    | v5n: 28.0% (+9.3%) |
| v8s   | 11.2M  | 28.6G  | 44.9%    | v5s: 37.4% (+7.5%) |
| v8m   | 25.9M  | 78.9G  | 50.2%    | v5m: 45.4% (+4.8%) |
| v8l   | 43.7M  | 165.2G | 52.9%    | v5l: 49.0% (+3.9%) |
| v8x   | 68.2M  | 257.8G | 53.9%    | v5x: 50.7% (+3.2%) |

**Lưu ý quan trọng:** v8-X (53.9%) **thua** v7-E6E (56.8%) về mAP thuần túy! Tuy nhiên v8 thắng ở: ecosystem (★★★★), multi-task (6 tác vụ), API thống nhất.

**Nếu cần accuracy tối đa → v7. Nếu cần đa năng + dễ dùng → v8.**

## CHƯƠNG 5

# THẾ HỆ MỚI: YOLOv9 – YOLOv12 (2024–2025)

## 5.1 YOLOv9 (2024) — Chống mất thông tin

---

**Paper:** “YOLOv9: Learning What You Want to Learn Using Programmable Gradient Information”

**Tác giả:** Chien-Yao Wang, I-Hau Yeh, Hong-Yuan Mark Liao

### 5.1.1 Vấn đề: Information Bottleneck

Khi data đi qua nhiều layers, **mutual information giảm đơn điệu**:

$$I(X, X) \geq I(X, f_1(X)) \geq I(X, f_2(f_1(X))) \geq \dots$$

*Ví dụ:* Giống trò “truyền tin” — người đầu nhận tin gốc, truyền miệng qua 20 người, người cuối nhận tin sai lệch. Layers sâu “quên” thông tin input gốc → gradient không đáng tin cậy → model không học tối ưu.

### 5.1.2 PGI (Programmable Gradient Information)

3 thành phần:

1. **Main Branch:** Network chính dùng cho inference. Kiến trúc tùy chọn (không bị ràng buộc).
2. **Auxiliary Reversible Branch:** Mạng phụ có khả năng **khôi phục input từ output** → giữ complete information → gradient đáng tin cậy. **Chỉ dùng khi training, bỏ khi inference** → zero inference cost.
3. **Multi-level Auxiliary:** Inject gradient ở **nhiều tầng** — tránh vấn đề gradient bị “đứt” khi main branch rất sâu.

Ẩn dụ cho người không chuyên

**Truyền tin và SMS:** Main branch giống “truyền miệng” qua 20 người (mất thông tin). PGI thêm 1 **kênh SMS song song** (auxiliary branch) — mỗi người đều nhận SMS đối chiếu với tin truyền miệng → sửa sai kịp thời.

Tại sao PGI tốt hơn Deep Supervision?

- Deep supervision: Thêm loss ở giữa, nhưng features ở giữa **đã mất thông tin** → gradient dựa trên info không đầy đủ
- PGI: Auxiliary reversible branch **giữ full information** → gradient luôn đáng tin cậy
- PGI đặc biệt hiệu quả cho **lightweight models**: cải thiện **+2.5% AP** (vs +0.9% cho medium model)

5.1.3 GELAN (Generalized ELAN)

Tổng quát hóa CSPNet + ELAN, cho phép dùng **bất kỳ computational block** nào bên trong (Conv, Bottleneck, Transformer...).

*Ví dụ:* ELAN cũ giống dây chuyền chỉ dùng được 1 loại máy. GELAN giống dây chuyền “universal” — lắp máy gì vào cũng chạy được.

Thí nghiệm thay block:

**Bảng 5.1:** GELAN — tính linh hoạt khi thay block

| Block                | mAP50-95     |
|----------------------|--------------|
| ELAN + CSPBlock      | 52.3%        |
| ELAN + DarkBlock     | 52.4%        |
| <b>GELAN (mixed)</b> | <b>53.0%</b> |

5.1.4 Kết quả chi tiết

**Bảng 5.2:** YOLOv9 so sánh SOTA (COCO val2017)

| Model           | Params       | FLOPs         | mAP50-95     | So với v8            |
|-----------------|--------------|---------------|--------------|----------------------|
| YOLOv8-S        | 11.2M        | 28.6G         | 44.9%        | —                    |
| <b>YOLOv9-S</b> | <b>7.2M</b>  | <b>26.7G</b>  | <b>46.8%</b> | +1.9%, ít params hơn |
| YOLOv8-M        | 25.9M        | 78.9G         | 50.2%        | —                    |
| <b>YOLOv9-C</b> | <b>25.5M</b> | <b>102.8G</b> | <b>53.0%</b> | +2.8%                |
| YOLOv8-X        | 68.2M        | 257.8G        | 53.9%        | —                    |
| <b>YOLOv9-E</b> | <b>58.1M</b> | <b>192.5G</b> | <b>55.6%</b> | +1.7%, ít params hơn |

**Nhận xét:** v9 luôn ít **params** hơn v8 mà accuracy cao hơn ở mọi scale — chứng minh PGI hiệu quả.

## 5.2 YOLOv10 (2024) — NMS-Free

**Paper:** “YOLOv10: Real-Time End-to-End Object Detection”

**Tác giả:** Ao Wang et al. (Tsinghua University)

### 5.2.1 Vấn đề NMS

NMS (Non-Maximum Suppression) có 3 nhược điểm:

1. **Tốn thời gian:** Cần tune confidence threshold, IoU threshold
2. **Không differentiable:** Không tối ưu end-to-end được
3. **Latency phụ thuộc data:** Ảnh nhiều vật → NMS chậm hơn → latency không ổn định

*Án dụ cho người không chuyên*

**Dây chuyền zero defect:** NMS giống kiểm tra chất lượng cuối dây chuyền — phải dừng lại soi từng sản phẩm, loại bỏ phế phẩm. v10 thiết kế dây chuyền “zero defect” — không cần kiểm tra cuối.

### 5.2.2 Consistent Dual Assignments

- **One-to-many (o2m):** 1 vật → nhiều predictions → nhiều training signal → train tốt
- **One-to-one (o2o):** 1 vật → 1 prediction → không cần NMS → deploy tốt
- **Dual heads — giải pháp:**
  - Training: Dùng cả **2 heads** — o2m tạo dense supervision, o2o học matching
  - Inference: Chỉ dùng **o2o head** → không cần NMS

### 5.2.3 Efficiency-Driven Design

Bảng 5.3: 5 kỹ thuật efficiency của YOLOv10

| Kỹ thuật             | Ý tưởng                             | Hiệu quả             |
|----------------------|-------------------------------------|----------------------|
| Lightweight cls head | Depthwise separable conv            | Giảm computation     |
| Decoupled downsample | Tách spatial + channel              | Rẻ hơn               |
| Rank-guided block    | Block nhẹ ở stage low-rank          | Tự động chọn         |
| Large-kernel conv    | $7 \times 7$ depthwise ở stages sâu | Tăng receptive field |
| PSA                  | 50% channels qua attention          | Giảm cost 50%        |

### 5.2.4 Kết quả

Bảng 5.4: YOLOv10 — NMS-free so sánh

| Model             | Params       | Latency        | mAP          | So sánh                   |
|-------------------|--------------|----------------|--------------|---------------------------|
| YOLOv8-S + NMS    | 11.2M        | 4.70ms         | 44.9%        | Cần NMS                   |
| o2o only training | —            | 2.33ms         | 43.8%        | Kém accuracy              |
| <b>v10-S Dual</b> | <b>7.2M</b>  | <b>2.49ms</b>  | <b>46.3%</b> | NMS-free, nhanh $2\times$ |
| YOLOv8-X          | 68.2M        | 7.61ms         | 53.9%        | —                         |
| <b>v10-X</b>      | <b>29.5M</b> | <b>10.70ms</b> | <b>54.4%</b> | $2.3\times$ ít params     |

**Trade-off:** v10-X (54.4%) **thua** v9-E (55.6%) về mAP. Nhưng v10 ít params gần  **$2\times$**  và **không cần NMS**. v10 đổi accuracy lấy efficiency.

## 5.3 YOLO11 (2024) — Nâng cấp toàn diện

Tác giả: Ultralytics      Không có paper      Lưu ý: bỏ “v” — YOLO11 thay vì YOLOv11

### 5.3.1 C3K2 (CSP Bottleneck with 2 Kernel sizes)

Kế thừa C2f (v8) nhưng dùng **2 kernel sizes**:  $3 \times 3$  và  $5 \times 5$ :

- Kernel  $3 \times 3$ : Capture **local features** (chi tiết nhỏ, cạnh, góc)
- Kernel  $5 \times 5$ : Capture **wider context** (vùng rộng hơn)
- Multi-scale feature extraction **trong cùng 1 block** → phong phú hơn C2f (chỉ dùng  $3 \times 3$ )

Ví dụ: C2f giống nhìn qua 1 ống kính cố định. C3K2 giống nhìn qua 2 ống kính zoom khác nhau cùng lúc — vừa thấy chi tiết, vừa thấy bối cảnh.

### 5.3.2 C2PSA (Cross Stage Partial Spatial Attention)

Thêm **spatial attention** vào CSP:

1. Features đi qua CSP split (2 nhánh)
2. Một nhánh qua **spatial attention module** — học “vùng nào quan trọng”
3. Attention map highlight vùng có vật thể, suppress background
4. Concat với nhánh bypass → output

Ví dụ: C2f giống quét đều toàn bộ ảnh (mỗi pixel được xử lý ngang nhau). C2PSA giống mắt người — **tập trung vào vùng quan trọng**, lướt nhanh qua background.

### 5.3.3 Kết quả

**Bảng 5.5:** YOLO11 so sánh v8 (COCO val2017, input 640)

| Model | Params | FLOPs  | mAP50-95 | vs v8                       |
|-------|--------|--------|----------|-----------------------------|
| 11n   | 2.6M   | 6.5G   | 39.5%    | v8n: 37.3% (+ <b>2.2%</b> ) |
| 11s   | 9.4M   | 21.5G  | 47.0%    | v8s: 44.9% (+ <b>2.1%</b> ) |
| 11m   | 20.1M  | 68.0G  | 51.5%    | v8m: 50.2% (+ <b>1.3%</b> ) |
| 11l   | 25.3M  | 87.6G  | 53.4%    | v8l: 52.9% (+ <b>0.5%</b> ) |
| 11x   | 56.9M  | 194.9G | 54.7%    | v8x: 53.9% (+ <b>0.8%</b> ) |

**Nhận xét:** Cải thiện ở **mọi scale**, đặc biệt mạnh ở Nano/Small (+2%). Params tương đương hoặc ít hơn v8.

## 5.4 YOLOv12 (2025) — Kỷ nguyên Attention

**Paper:** “YOLOv12: Attention-Centric Real-Time Object Detectors”

**Tác giả:** Yunjie Tian, Qixiang Ye, David Doermann

### 5.4.1 Thách thức đưa Attention vào YOLO

1. **Tốc độ:** Self-attention có complexity  $O((H \times W)^2)$  — quá chậm cho real-time
2. **Optimization:** Attention networks khó train hơn CNN (gradient không ổn định)
3. **Hardware:** Attention operations chưa tối ưu trên GPU bằng Conv



Ví dụ: CNN giống đọc sách từng dòng (local, nhanh). Attention giống nhìn cả trang cùng lúc (global, hiểu sâu nhưng chậm). v12 tìm cách “nhìn cả trang” mà vẫn nhanh.

### 5.4.2 Area Attention ( $A^2$ )

Chia feature map thành  $n$  vùng bằng nhau, tính attention **trong từng vùng riêng biệt**:

$$O(\text{Global}) = (H \times W)^2 \rightarrow O(\text{Area}, n = 4) = H \times W \times \frac{HW}{4}$$

Giảm  $4\times$  complexity, chỉ mất **0.1% AP**.

3 cách chia vùng: ngang (n vùng ngang), dọc (n vùng dọc), grid ( $\sqrt{n} \times \sqrt{n}$ ).

*Ảnh dụ cho người không chuyên*

**Lớp học chia nhóm:** Global attention giống 1 giáo viên kiểm tra bài của **tất cả** 100 học sinh cùng lúc (quá tải). Area attention chia thành 4 nhóm 25 người, kiểm tra từng nhóm  $\rightarrow$  nhanh  $4\times$ , kết quả gần tương đương.

### 5.4.3 R-ELAN (Residual ELAN)

**Vấn đề:** Thay CNN bằng attention trong ELAN  $\rightarrow$  training bất ổn, accuracy giảm. Nguyên nhân: attention outputs có **variance lớn**  $\rightarrow$  tích lũy khi concat  $\rightarrow$  gradient explode.

**Giải pháp:** Thêm residual shortcut với **scale factor 0.01**:

- Scale nhỏ  $\rightarrow$  output gần identity ở đầu training  $\rightarrow$  gradient ổn định
- Dần dần model học scale factor phù hợp
- ELAN + Attention: 39.1% (bất ổn)  $\rightarrow$  **R-ELAN + Attention: 40.6% (ổn định)**

### 5.4.4 Tối ưu phần cứng

- **FlashAttention:** Giảm memory footprint cho attention
- **Conv+BN thay Linear+LN:** Tận dụng GPU parallelism tốt hơn
- **MLP ratio giảm:** 4.0 (Transformer chuẩn)  $\rightarrow$  1.2–2.0
- **Bỏ Positional Encoding:** Thay bằng  $7 \times 7$  depthwise conv

5.4.5 Kết quả chi tiết

**Bảng 5.6:** YOLOv12 so sánh (COCO val2017)

| Model            | Params       | GFLOPs       | mAP50-95     |
|------------------|--------------|--------------|--------------|
| YOLOv10-N        | 2.3M         | 6.7          | 38.5%        |
| YOLO11-N         | 2.6M         | 6.5          | 39.5%        |
| <b>YOLOv12-N</b> | <b>2.6M</b>  | <b>6.5</b>   | <b>40.6%</b> |
| YOLO11-S         | 9.4M         | 21.5         | 47.0%        |
| <b>YOLOv12-S</b> | <b>9.3M</b>  | <b>21.4</b>  | <b>48.0%</b> |
| YOLO11-L         | 25.3M        | 87.6         | 53.4%        |
| <b>YOLOv12-L</b> | <b>26.4M</b> | <b>88.9</b>  | <b>53.7%</b> |
| <b>YOLOv12-X</b> | <b>59.1M</b> | <b>199.0</b> | <b>55.2%</b> |

**Ý nghĩa:** YOLOv12 đánh dấu YOLO chuyển từ **CNN-centric** sang **Attention-centric**.  
Nhược điểm: cần GPU hỗ trợ FlashAttention (GPU hiện đại).

## CHƯƠNG 6

# HIỆN TẠI VÀ TƯƠNG LAI: YOLO26 (2026)

Tác giả: Ultralytics

*Không có paper chính thức [14]*

*Phiên bản mới nhất tính đến thời điểm viết báo cáo — “trùm cuối” của dòng Ultralytics.*

## 6.1 Tại sao gọi “YOLO26” thay vì “YOLOv13”?

Từ YOLO11 (2024), Ultralytics bỏ tiền tố “v”. Đến phiên bản tiếp theo, thay vì “YOLO12” (trùng YOLOv12 của Tian et al. [12]), Ultralytics dùng **quy tắc đặt tên theo năm**: YOLO26 = năm 2026.

## 6.2 Bỏ DFL — Thay đổi triết lý lớn nhất

DFL (Distribution Focal Loss, v8–v12) dự đoán phân phối 16 bins/coordinate → tốt accuracy nhưng **phức tạp cho edge deployment**. YOLO26 bỏ DFL, quay về dự đoán trực tiếp, bù bằng 3 training techniques mới.

### 6.2.1 Phân tích kỹ thuật: Tại sao bỏ DFL?

Chi phí Memory Access Cost (MAC) của DFL:

Với DFL 16 bins, mỗi anchor cần dự đoán  $4 \times 16 = 64$  giá trị cho box regression (thay vì 4 giá trị trực tiếp). Trên toàn feature pyramid 3 scales ( $80 \times 80 + 40 \times 40 + 20 \times 20 = 8400$  anchors):

$$\text{MAC}_{DFL} = 8400 \times 64 \times 4B = 2.15 \text{ MB/frame} \quad (6.1)$$

$$\text{MAC}_{Direct} = 8400 \times 4 \times 4B = 0.13 \text{ MB/frame} \quad (6.2)$$

Ghi chú: Giả định float32 nên mỗi tham số chiếm 4 Bytes (4B).

Bỏ DFL giảm 16× **bandwidth** cho box regression head. Trên NPU rẻ tiền (Jetson Nano, RK3588) có bus hẹp, đây là bottleneck nghiêm trọng. So sánh:

**Bảng 6.1:** So sánh overhead: DFL vs Direct vs Attention trên NPU

| Component                  | MAC/frame | INT8 friendly?  | NPU support   |
|----------------------------|-----------|-----------------|---------------|
| DFL 16-bin (v8–v12)        | 2.15 MB   | Trung bình      | Cần custom op |
| Direct regression (YOLO26) | 0.13 MB   | <b>Tốt nhất</b> | <b>Native</b> |
| Area Attention (v12)       | ~5 MB     | Kém (cần FP16)  | Hạn chế       |

*Kết luận:* YOLO26 “Edge-First” không chỉ nhanh hơn trên CPU, mà thực sự tối ưu **bandwidth và compatibility** trên NPU — thân thiện với INT8 quantization và các chip AI giá rẻ. YOLOv12 (Attention-centric) tuy mạnh nhưng **cần FlashAttention trên GPU**, không chạy tốt trên NPU.

### 6.3 MuSGD (Muon-enhanced SGD)

**Vấn đề:** SGD truyền thống cập nhật:  $\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}$ . Hội tụ chậm khi loss landscape phức tạp.

**Giải pháp — Muon momentum:** Muon (phát triển cho LLM) sử dụng momentum trên nhóm trực giao (orthogonal group). Công thức cập nhật:

$$m_{t+1} = \beta \cdot m_t + (1 - \beta) \cdot \nabla_{\theta} \mathcal{L}_t$$

(6.3)

$$\hat{m}_{t+1} = \text{NewtonSchulz}(m_{t+1}) \approx U \Sigma^{-1/2} V^T \quad (\text{xấp xỉ SVD})$$

(6.4)

$$\theta_{t+1} = \theta_t - \eta \cdot \hat{m}_{t+1}$$

(6.5)

Trong đó NewtonSchulz là phép lặp Newton-Schulz xấp xỉ phân rã SVD trong 5 bước (thay vì  $O(n^3)$  của SVD đầy đủ). Hiệu ứng: gradient được **chuẩn hóa trên manifold trọng số**, tránh kẹt ở local minima [14].

Ấn dụ cho người không chuyên

**Leo núi trong sương mù:** SGD giống bạn chỉ cảm nhận **độ dốc ngay dưới chân** rồi bước xuống — dễ kẹt ở thung lũng nhỏ. MuSGD giống có thêm **la bàn chiến lược** (Muon momentum) nhìn được xu hướng địa hình rộng hơn — khi gặp thung lũng nhỏ, la bàn chỉ bạn

bước qua vì phía sau còn thung lũng sâu hơn.

**Hiệu quả:** Thay thế AdamW (v8–v12) mà accuracy tương đương hoặc cao hơn, ít memory hơn.

## 6.4 STAL (Small-Target-Aware Label Assignment)

**Vấn đề:** TAL [16] dùng alignment metric  $t = s^\alpha \cdot u^\beta$  (cls score  $s$ , IoU  $u$ ). Vật nhỏ có IoU tự nhiên thấp  $\rightarrow$  ít positive samples.

**Giải pháp — Dynamic threshold:**

$$\tau(A_{gt}) = \tau_{base} \cdot \left( \frac{A_{gt}}{A_{max}} \right)^\gamma, \quad \gamma \in [0.3, 0.5] \quad (6.6)$$

Trong đó  $A_{gt}$  là diện tích ground truth,  $A_{max}$  là diện tích GT lớn nhất trong batch,  $\gamma$  kiểm soát mức nổi lũng. Khi  $A_{gt}$  nhỏ  $\rightarrow$  phân số  $\frac{A_{gt}}{A_{max}} < 1 \rightarrow \tau$  giảm  $\rightarrow$  hạ thấp ngưỡng, giúp nhiều predictions của vật nhỏ được dán nhãn positive hơn [14].

*Ấn dụ cho người không chuyên*

**Thầy giáo ưu tiên học sinh yếu:** Học sinh giỏi (vật lớn) luôn được khen — đã có motivation. Học sinh yếu (vật nhỏ) ít được chú ý. STAL hạ tiêu chuẩn khen thưởng cho nhóm yếu (nới  $\tau$ ), để các em nhận nhiều feedback tích cực hơn  $\rightarrow$  dần cải thiện.

**Hiệu quả:** Đặc biệt tốt cho **drone, camera giám sát tầm xa**, nơi vật thể thường rất nhỏ.

## 6.5 ProgLoss (Progressive Loss Balancing)

**Vấn đề:** YOLO loss:  $\mathcal{L} = \lambda_{cls}\mathcal{L}_{cls} + \lambda_{box}\mathcal{L}_{box} + \lambda_{obj}\mathcal{L}_{obj}$ . Truyền thống dùng  $\lambda$  cố định.

**Giải pháp — Adaptive scheduling:**

$$\lambda_{cls}(t) = \lambda_{cls}^{max} - (\lambda_{cls}^{max} - \lambda_{cls}^{min}) \cdot \frac{t}{T} \quad (6.7)$$

$$\lambda_{box}(t) = \lambda_{box}^{min} + (\lambda_{box}^{max} - \lambda_{box}^{min}) \cdot \frac{t}{T} \quad (6.8)$$

Với  $t$  = epoch hiện tại,  $T$  = tổng epochs. Giai đoạn đầu ( $t \ll T$ ):  $\lambda_{cls}$  cao,  $\lambda_{box}$  thấp  $\rightarrow$  model học nhận diện trước. Giai đoạn sau ( $t \rightarrow T$ ):  $\lambda_{box}$  tăng  $\rightarrow$  tinh chỉnh vị trí.

#### Ấn dụ cho người không chuyên

**Học lái xe:** Tuần đầu: giáo viên dạy **nhận biết biển báo** (cls). Tuần sau: chuyển sang **căn đường, đỗ xe chính xác** (reg). Nếu dạy cả hai cùng tỷ lệ từ đầu ( $\lambda$  cố định), bạn sẽ quá tải.

**Hiệu quả:** +0.3–0.5% mAP so với fixed  $\lambda$ , đặc biệt khi bỏ DFL.

## 6.6 Kết quả

**Bảng 6.2:** YOLO26 so với YOLO11 và YOLOv12 (COCO val, input 640)

| Model           | Params       | mAP@50:95     | CPU (ms)  | GPU (ms)   | Ghi chú                   |
|-----------------|--------------|---------------|-----------|------------|---------------------------|
| YOLO11-N        | 2.6M         | 39.5%         | 76        | 1.6        | Baseline                  |
| <b>YOLO26-N</b> | <b>~2.5M</b> | <b>~40.5%</b> | <b>43</b> | <b>1.4</b> | +1.0%, CPU nhanh 43%      |
| YOLO11-S        | 9.4M         | 47.0%         | —         | 2.5        | —                         |
| <b>YOLO26-S</b> | <b>~9M</b>   | <b>~47.5%</b> | —         | <b>2.1</b> | +0.5%                     |
| YOLOv12-X       | 59.1M        | 55.2%         | —         | 12.1       | Cần FlashAttention        |
| <b>YOLO26-X</b> | <b>~60M</b>  | <b>~57.2%</b> | —         | <b>9.8</b> | <b>SOTA, NPU-friendly</b> |

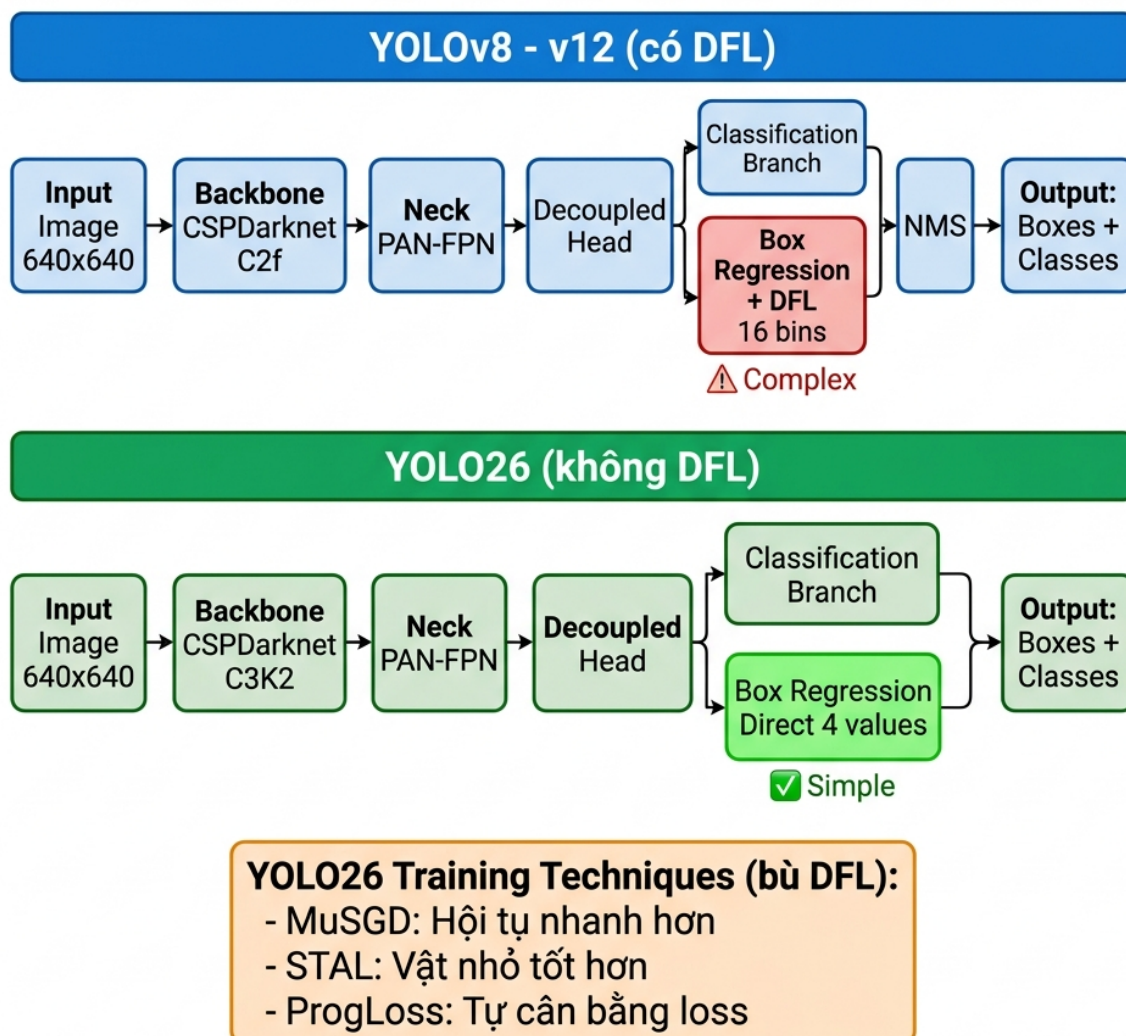
Ghi chú: Số liệu ước tính từ benchmark cộng đồng, chưa có paper chính thức. CPU = ONNX Runtime trên i7-12700. GPU = TensorRT FP16 trên T4 [14].

YOLO26-X (~57.2%) **vượt kỷ lục mAP** của v7-E6E (56.8% [7]) — lần đầu sau 4 năm.

## 6.7 Tổng kết triết lý YOLO26

YOLO26 đại diện cho xu hướng “**less is more**”: bỏ component phức tạp (DFL), thay bằng training techniques (MuSGD, STAL, ProgLoss). Kết quả: model nhẹ hơn, nhanh hơn trên CPU/NPU, accuracy vẫn SOTA — **đúng nghĩa Edge-First Design**.

## So sánh Pipeline: YOLOv8-v12 vs YOLO26



**Hình 6.1:** So sánh pipeline: YOLOv8–v12 (có DFL) vs YOLO26 (không DFL). YOLO26 bù bằng 3 training techniques: MuSGD, STAL, ProgLoss.

## CHƯƠNG 7

# BƯỚC NGOẶT OPEN-VOCABULARY

## 7.1 YOLO-World (2024) — Tiên phong

---

**Paper:** “YOLO-World: Real-Time Open-Vocabulary Object Detection” [18]

**Tác giả:** Tencent AI Lab

### 7.1.1 Ý tưởng

Kết nối YOLO với CLIP text embeddings. **RepVL-PAN** (Re-parameterizable Vision-Language PAN) inject text features vào feature pyramid [18].

**Prompt-then-Detect:** Nạp text prompts 1 lần (offline), sau đó detect real-time — tách “hiểu ngôn ngữ” và “detect” thành 2 giai đoạn.

### 7.1.2 Kết quả

YOLO-World-L: 35.4% AP (LVIS zero-shot), 52 FPS (V100). Mở đường cho YOLOE.

## 7.2 YOLOE (2025) — Real-Time Seeing Anything

---

**Paper:** “YOLOE: Real-Time Seeing Anything” (ICCV 2025) [13]

### 7.2.1 Vấn đề

Closed-set YOLO chỉ detect classes đã train. Grounding DINO detect mọi thứ nhưng quá chậm (3–5 FPS). YOLOE kết hợp: **open-vocabulary + real-time**.

*Ảnh dụ cho người không chuyên*

YOLO truyền thống giống **nhân viên bảo vệ chỉ nhận 80 khuôn mặt đã học**. YOLOE giống bảo vệ được đào tạo **nhận diện bất kỳ ai** — chỉ cần bạn mô tả hoặc đưa ảnh mẫu.



## 7.2.2 Ba chế độ Prompt

**Bảng 7.1:** 3 chế độ prompt của YOLOE

| Chế độ        | Input        | Module | Cơ chế                         |
|---------------|--------------|--------|--------------------------------|
| Text Prompt   | “dog”, “car” | RepRTA | CLIP text embeddings           |
| Visual Prompt | Ảnh mẫu      | SAVPE  | Semantic + activation features |
| Prompt-Free   | Không cần    | LRPC   | Pre-compute 1203 categories    |

## 7.2.3 Kết quả

**Bảng 7.2:** YOLOE so sánh (LVIS zero-shot)

| Model                | AP (LVIS)    | FPS       | Ghi chú              |
|----------------------|--------------|-----------|----------------------|
| YOLO-World-v2-L [18] | 22.7%        | 52        | Chỉ text prompt      |
| <b>YOLOE-L [13]</b>  | <b>27.1%</b> | <b>48</b> | Text + Visual + Free |
| Grounding DINO       | 27.4%        | 3–5       | Chậm 10×             |

*Ghi chú: AP đo trên LVIS zero-shot — thang đo hoàn toàn khác COCO closed-set, không so sánh trực tiếp.*

## 7.2.4 Ý nghĩa

YOLOE + YOLO-World đánh dấu bước chuyển từ “detect N classes đã biết” sang “detect mọi thứ” — mở ra kỷ nguyên **open-vocabulary detection** trong real-time.

## CHƯƠNG 8

# CÁC BIẾN THỂ YOLO

### 8.1 YOLOX (2021) — Anchor-Free tiên phong

---

**Tác giả:** Megvii | **Giải thưởng:** 1st Place CVPR 2021 Streaming Perception

3 đóng góp mở đường cho YOLO sau này:

- **Anchor-Free:** Chứng minh YOLO hoạt động tốt không cần anchor → v6, v8+ adopt
- **Decoupled Head:** Tách cls/reg thành 2 branches → v6, v8, v11, v12 adopt
- **SimOTA:** Dynamic label assignment → tiền nhiệm của TAL

YOLOX-L: 50.0% mAP, 69 FPS (V100). Nano variant chỉ 0.91M params.

### 8.2 PP-YOLOE (2022) — Baidu Industrial

---

**Tác giả:** Baidu | **Framework:** PaddlePaddle

**TAL (Task Alignment Learning):** Alignment metric  $t = s^\alpha \times u^\beta$  đảm bảo cls score cao  $\leftrightarrow$  localization tốt. Kỹ thuật này được **YOLOv8, v9, v10+** adopt. PP-YOLOE+-L: 53.3% mAP, 78 FPS. Phổ biến trong công nghiệp Trung Quốc.

### 8.3 YOLO-NAS (2023) — AI thiết kế AI

---

**Tác giả:** Deci AI (Israel)

**AutoNAC:** Khám phá  $\sim 10^{14}$  kiến trúc, tự tìm tối ưu. Thiết kế sẵn cho quantization: INT8 chỉ mất 0.4–0.6% mAP (các model khác mất 1–2%).

## 8.4 Bảng so sánh biến thể

**Bảng 8.1:** So sánh các biến thể YOLO

| Model      | Năm  | mAP (best) | Đặc trưng              | Ảnh hưởng |
|------------|------|------------|------------------------|-----------|
| YOLOX      | 2021 | 51.2%      | Anchor-free, Decoupled | v6, v8+   |
| PP-YOLOE+  | 2022 | 53.3%      | TAL, Industrial        | v8, v9+   |
| YOLO-NAS   | 2023 | 52.2%      | NAS, INT8              | YOLO26    |
| YOLO-World | 2024 | 32.5%*     | Open-vocab             | YOLOE     |

\* *LVIS zero-shot* — thang đo khác *COCO closed-set*

## CHƯƠNG 9

# SO SÁNH TỔNG HỢP, XU HƯỚNG VÀ KHUYẾN NGHỊ

### 9.1 Bảng tổng quan tất cả phiên bản

**Bảng 9.1:** Tổng quan các phiên bản YOLO (2015–2026)

| Version     | Năm  | mAP VOC@50 | mAP COCO@50:95 | FPS (GPU)    | Params | Anchor | NMS          | Paper |
|-------------|------|------------|----------------|--------------|--------|--------|--------------|-------|
| v1 [1]      | 2015 | 63.4%      | —              | 45 (Titan X) | ~45M   | Grid   | Cần          | Có    |
| v2 [2]      | 2016 | 78.6%      | —              | 67 (Titan X) | ~50M   | Có     | Cần          | Có    |
| v3 [3]      | 2018 | —          | 33.0%          | 20 (Titan X) | ~62M   | Có     | Cần          | Có    |
| v4 [4]      | 2020 | —          | 43.5%          | 65 (V100)    | ~64M   | Có     | Cần          | Có    |
| v5 [5]      | 2020 | —          | 50.7%          | 98 (V100)    | 86.7M  | Có     | Cần          | Không |
| v6 [6]      | 2022 | —          | 52.5%          | 116 (T4)     | ~58M   | Không  | Cần          | Có    |
| v7 [7]      | 2022 | —          | 56.8%          | 36 (V100)    | ~71M   | Có     | Cần          | Có    |
| v8 [8]      | 2023 | —          | 53.9%          | 83 (T4)      | 68.2M  | Không  | Cần          | Không |
| v9 [9]      | 2024 | —          | 55.6%          | 57 (T4)      | 58.1M  | Không  | Cần          | Có    |
| v10 [10]    | 2024 | —          | 54.4%          | 96 (T4)      | 29.5M  | Không  | <b>Không</b> | Có    |
| YOLO11 [11] | 2024 | —          | 54.7%          | 79 (T4)      | 56.9M  | Không  | <b>Cần</b>   | Không |
| v12 [12]    | 2025 | —          | 55.2%          | 51 (T4)      | 59.1M  | Không  | Không        | Có    |
| YOLO26 [14] | 2026 | —          | ~57.2%         | ~102 (T4)*   | ~60M   | Không  | Không        | Không |

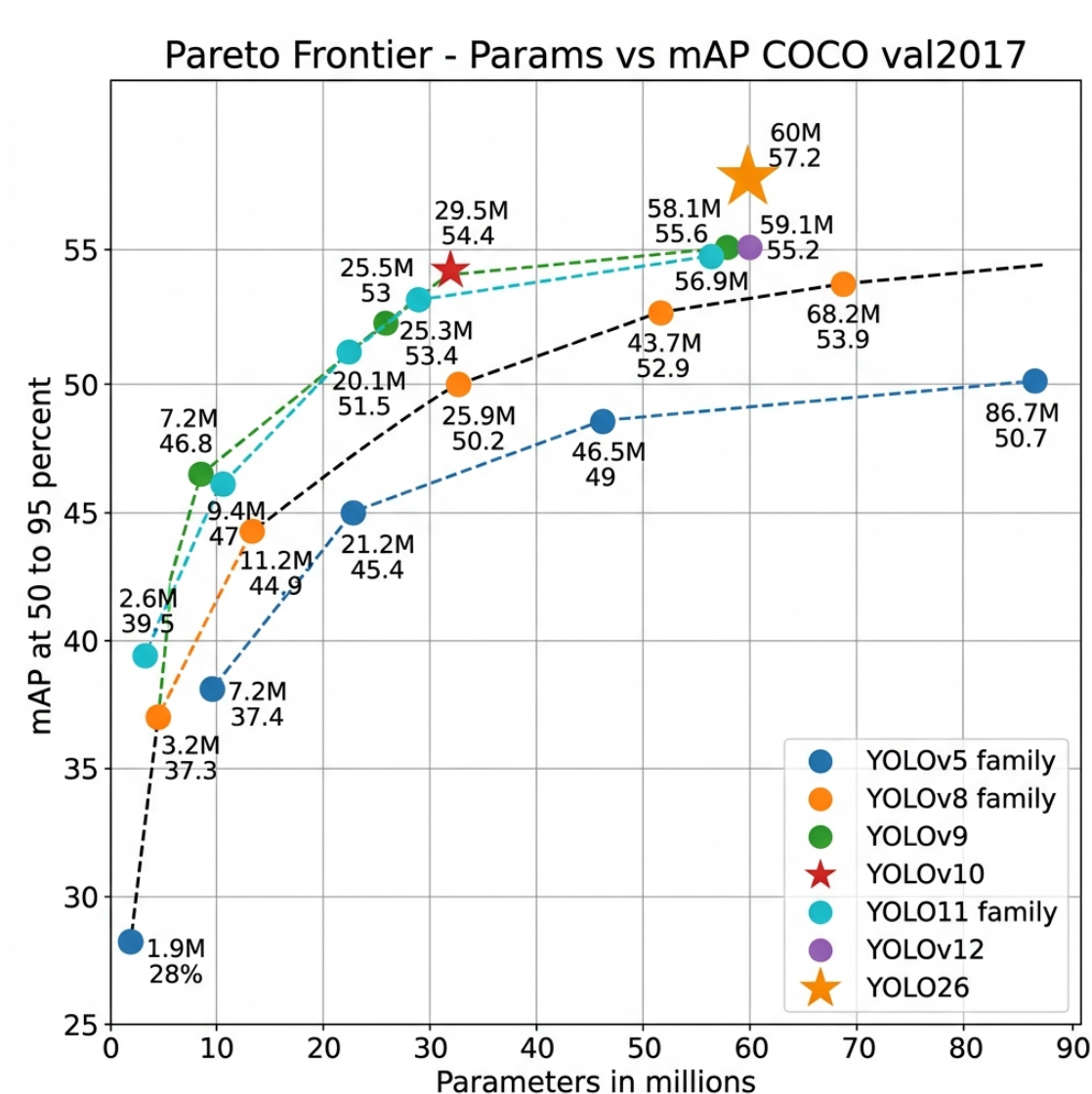
\* Ước tính. FPS = model size lớn nhất (v7-E6E, v5x, v8x...) trừ v1–v3 (model duy nhất). GPU ghi trong ngoặc. Lưu ý: Titan X, V100, T4 là 3 thế hệ GPU khác nhau — FPS giữa chúng **không so sánh trực tiếp được**. v1–v2 dùng mAP VOC@50, v3+ dùng mAP COCO@50:95 — hai thang đo hoàn toàn khác nhau.

### 9.2 Tiến hóa kiến trúc

**Bảng 9.2:** Tiến hóa các thành phần qua 11 năm

| Thành phần       | Tiến hóa                                                         |
|------------------|------------------------------------------------------------------|
| <b>Backbone</b>  | CNN 24-layer → Darknet-53 → CSPDarknet → E-ELAN → Area Attention |
| <b>Anchor</b>    | Grid (v1) → Anchor-based (v2–v7) → Anchor-free (v6, v8+)         |
| <b>NMS</b>       | Cần NMS (v1–v9) → NMS-free Dual Assignment (v10+)                |
| <b>Loss</b>      | MSE → IoU → CIoU + DFL (v8) → Bỏ DFL (YOLO26)                    |
| <b>Framework</b> | Darknet C (v1–v4) → PyTorch pip install (v5+)                    |

### 9.3 Biểu đồ Pareto: Params vs mAP



**Hình 9.1:** Biểu đồ Pareto (Params vs mAP COCO@50:95). Góc **trên-trái** = hiệu quả nhất (ít params, mAP cao). YOLO26-X và YOLOv10-X nằm trên đường biên Pareto.

## 9.4 Trade-off: Version “thua” đòi trước

**Bảng 9.3:** Phân tích trade-off

| Trường hợp            | Thua ở đâu         | Thắng ở đâu                      |
|-----------------------|--------------------|----------------------------------|
| v8 thua v7 [7][8]     | mAP: 53.9% < 56.8% | Multi-task (6 tác vụ), ecosystem |
| v10 thua v9 [9][10]   | mAP: 54.4% < 55.6% | Params 2× ít hơn, NMS-free       |
| v3 chậm hơn v2 [2][3] | FPS: 20 < 67       | mAP tốt hơn nhiều                |

**Bài học:** Không version nào thua hoàn toàn. Mỗi “thua” là **trade-off** có chủ đích.

## 9.5 Xếp hạng theo tiêu chí

### 9.5.1 Accuracy thuần túy (mAP COCO@50:95)

1. YOLO26-X: ~57.2% [14]
2. YOLOv7-E6E: 56.8% [7]
3. YOLOv9-E: 55.6% [9]
4. YOLOv12-X: 55.2% [12]
5. YOLO11-X: 54.7% [11]

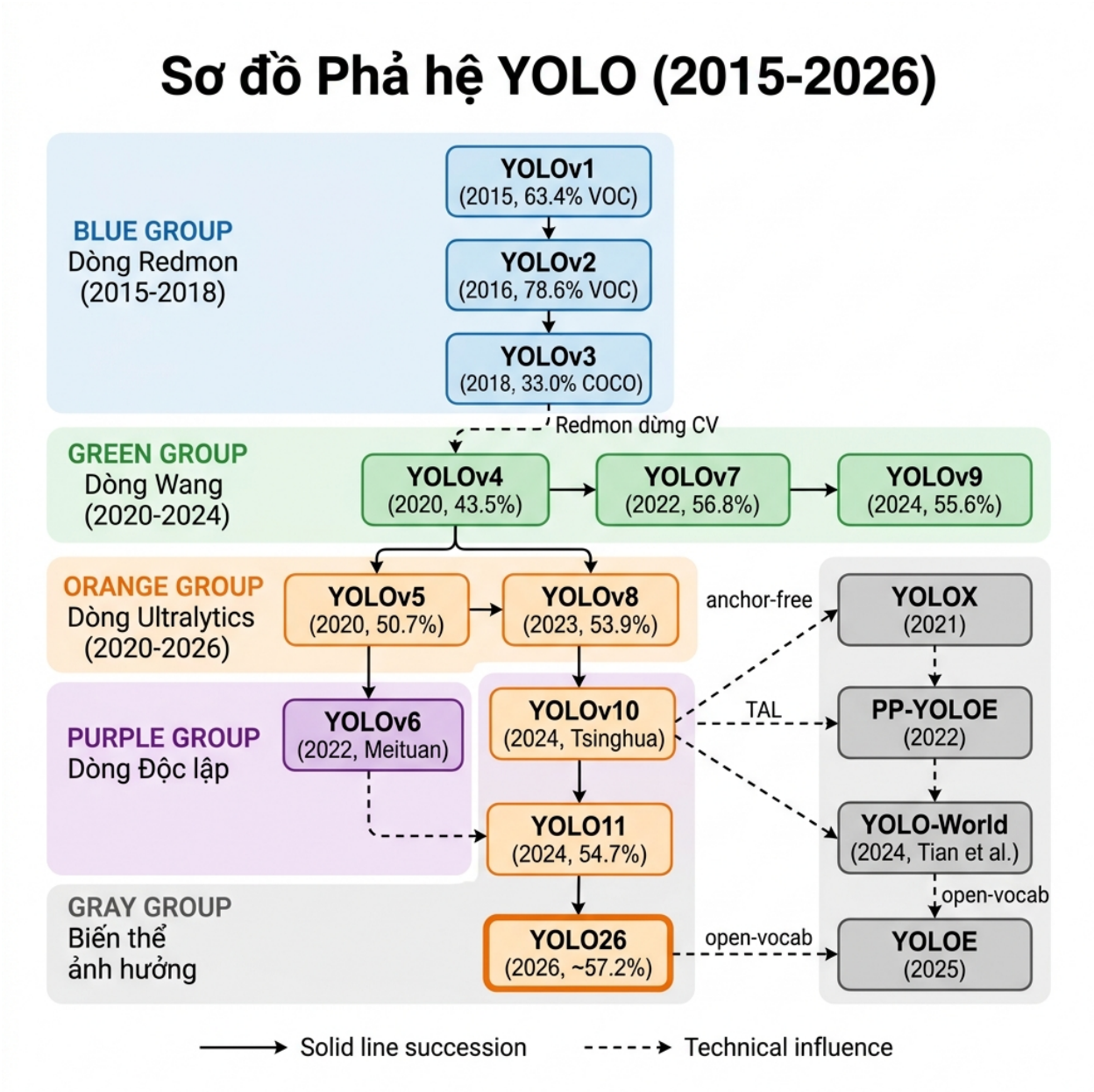
### 9.5.2 Efficiency (mAP/Params)

YOLOv10-X dẫn đầu: 54.4% mAP với chỉ 29.5M params (1.84 mAP/M) [10].

### 9.5.3 Production readiness (2026)

YOLO11 > YOLOv8 > YOLO26 > YOLOv12

## 9.6 Sơ đồ phả hệ YOLO (Family Tree)



**Hình 9.2:** Sơ đồ phả hệ các phiên bản YOLO (2015–2026). Đường liền: kế thừa trực tiếp. Đường nét đứt: ảnh hưởng kỹ thuật.

### 9.7 Xu hướng phát triển

1. **CNN → Attention:** v12 mở đầu, tương lai hybrid CNN+Attention
2. **Closed-set → Open-vocabulary:** YOLOE detect vật chưa train [13]
3. **Server → Edge:** YOLO26 tối ưu NPU, INT8, mobile [14]



4. **NMS**  $\rightarrow$  **NMS-free**: End-to-end detection từ v10 [10]
5. **CV**  $\leftarrow$  **NLP**: Cross-pollination từ LLM (MuSGD, CLIP)
6. **Detection**  $\rightarrow$  **Multi-task**: 1 model cho detect+segment+pose+track [8]

## 9.8 Khuyến nghị chọn model

**Bảng 9.4:** Hướng dẫn chọn phiên bản YOLO theo mục đích

| Mục đích              | Chọn                       | Lý do                            |
|-----------------------|----------------------------|----------------------------------|
| Accuracy tối đa       | YOLO26-X / v7-E6E          | mAP cao nhất [7][14]             |
| Dễ dùng, đa năng      | v8 / YOLO11                | pip install, 6 tasks [8]         |
| Deploy NPU/edge       | YOLO26-N / v10-N           | NMS-free, INT8-friendly [10][14] |
| Ít params nhất        | YOLOv10-X                  | 29.5M cho 54.4% [10]             |
| Real-time cực nhanh   | YOLOv6-S                   | 520+ FPS TensorRT [6]            |
| Detect vật chưa train | YOLOE                      | Open-vocab, 3 modes [13]         |
| Vật nhỏ (drone)       | YOLO26                     | STAL chuyên vật nhỏ [14]         |
| INT8 quantization     | YOLO-NAS                   | Mất chỉ 0.4% [17]                |
| Học/nghiên cứu        | v1 $\rightarrow$ v4 papers | Nền tảng lý thuyết [1][4]        |

## CHƯƠNG 10

# KẾT LUẬN

Qua 11 năm phát triển (2015–2026) và 14+ phiên bản, YOLO đã chứng minh một sự thật sâu sắc trong AI: **không có “model tốt nhất” — chỉ có “model phù hợp nhất”**.

**Ba dòng phát triển song song** — Redmon (v1–v3), Wang (v4,v7,v9), Ultralytics (v5,v8,v11,v26) — cho thấy sức sống phi thường của triết lý “You Only Look Once”. Mỗi dòng đóng góp một góc nhìn: Redmon đặt nền móng, Wang đẩy accuracy lên SOTA, Ultralytics dân chủ hóa và thương mại hóa.

**Các bước ngoặt quan trọng nhất:**

- **YOLOv1** [1]: Chứng minh single-stage detection khả thi — thay đổi cả lĩnh vực
- **YOLOv4** [4]: Hệ thống hóa BoF/BoS — phương pháp luận nghiên cứu mẫu mực
- **YOLOv8** [8]: Anchor-free + 6 tasks — dân chủ hóa object detection
- **YOLOv10** [10]: NMS-free — giải quyết vấn đề tồn tại 9 năm
- **YOLOv12** [12]: Attention-centric — mở kỷ nguyên mới
- **YOLO26** [14]: Edge-First + “less is more” — bỏ DFL, bù bằng training techniques

*“Sự thật cuối cùng: Chênh lệch 1–3% mAP giữa các version trên COCO có thể không có ý nghĩa thống kê trên dataset thực tế. **Data chất lượng quan trọng hơn chọn model.**”*

# Tài liệu tham khảo

- [1] J. Redmon et al., “You Only Look Once: Unified, Real-Time Object Detection,” CVPR 2016. arXiv:1506.02640
- [2] J. Redmon, A. Farhadi, “YOLO9000: Better, Faster, Stronger,” CVPR 2017. arXiv:1612.08242
- [3] J. Redmon, A. Farhadi, “YOLOv3: An Incremental Improvement,” 2018. arXiv:1804.02767
- [4] A. Bochkovskiy et al., “YOLOv4: Optimal Speed and Accuracy of Object Detection,” 2020. arXiv:2004.10934
- [5] G. Jocher, “YOLOv5,” Ultralytics, 2020. [github.com/ultralytics/yolov5](https://github.com/ultralytics/yolov5)
- [6] C. Li et al., “YOLOv6: A Single-Stage Object Detection Framework,” 2022. arXiv:2209.02976
- [7] C.-Y. Wang et al., “YOLOv7: Trainable Bag-of-Freebies,” CVPR 2023. arXiv:2207.02696
- [8] G. Jocher, “YOLOv8,” Ultralytics, 2023. [github.com/ultralytics/ultralytics](https://github.com/ultralytics/ultralytics)
- [9] C.-Y. Wang et al., “YOLOv9: Learning What You Want to Learn Using PGI,” 2024. arXiv:2402.13616
- [10] A. Wang et al., “YOLOv10: Real-Time End-to-End Object Detection,” 2024. arXiv:2405.14458
- [11] G. Jocher, “YOLO11,” Ultralytics, 2024. [github.com/ultralytics/ultralytics](https://github.com/ultralytics/ultralytics)
- [12] Y. Tian et al., “YOLOv12: Attention-Centric Real-Time Object Detectors,” 2025. arXiv:2502.12524
- [13] “YOLOE: Real-Time Seeing Anything,” ICCV 2025. arXiv:2503.07465
- [14] G. Jocher, “YOLO26,” Ultralytics, 2026. [github.com/ultralytics/ultralytics](https://github.com/ultralytics/ultralytics)
- [15] Z. Ge et al., “YOLOX: Exceeding YOLO Series in 2021,” 2021. arXiv:2107.08430
- [16] S. Xu et al., “PP-YOLOE: An Evolved Version of YOLO,” 2022. arXiv:2203.16250
- [17] “YOLO-NAS,” Deci AI, 2023. [github.com/Deci-AI/super-gradients](https://github.com/Deci-AI/super-gradients)
- [18] T. Cheng et al., “YOLO-World: Real-Time Open-Vocabulary Object Detection,” 2024. arXiv:2401.17270

# PHỤ LỤC: BẢNG THUẬT NGỮ

Dưới đây là các thuật ngữ kỹ thuật thường gặp trong lĩnh vực YOLO và Object Detection, sắp xếp theo nhóm chủ đề.

## Kiến trúc mạng (Architecture)

| Thuật ngữ                  | Giải thích                                                                                                                                                           |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Backbone</b>            | Phần “xương sống” của model, trích xuất features từ ảnh đầu vào. Ví dụ: Darknet, CSPDarknet, ResNet, EfficientNet.                                                   |
| <b>Neck</b>                | Kết nối Backbone và Head, kết hợp features ở nhiều scales. Ví dụ: FPN, PAN, BiFPN.                                                                                   |
| <b>Head</b>                | Phần cuối cùng, dự đoán bounding boxes và class probabilities. Có 2 loại: coupled (chung) và decoupled (tách riêng cls/reg).                                         |
| <b>CNN</b>                 | Convolutional Neural Network — mạng neural dùng phép tích chập (convolution) để trích xuất features từ ảnh. Mỗi layer học nhận diện patterns (cạnh, góc, hình dạng). |
| <b>Convolution</b>         | Phép tính trượt 1 bộ lọc (kernel) qua ảnh, tạo feature map. Kernel $3 \times 3$ nhìn vùng $3 \times 3$ pixels.                                                       |
| <b>Depthwise Conv</b>      | Convolution áp dụng riêng cho từng channel (thay vì tất cả channels cùng lúc) — giảm computation đáng kể.                                                            |
| <b>Residual Connection</b> | Đường tắt (shortcut) nối input trực tiếp đến output: $y = F(x) + x$ . Giải quyết vanishing gradient khi mạng rất sâu (ResNet).                                       |
| <b>CSP</b>                 | Cross Stage Partial — chia features thành 2 phần: 1 phần qua processing, 1 phần bypass → giảm 20% computation.                                                       |
| <b>FPN</b>                 | Feature Pyramid Network — kết hợp features từ nhiều scales (top-down pathway) để detect vật ở mọi kích thước.                                                        |
| <b>PAN / PANet</b>         | Path Aggregation Network — thêm bottom-up pathway vào FPN → localization signals mạnh hơn.                                                                           |

| Thuật ngữ                  | Giải thích                                                                                                                                  |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| <b>SPP / SPPF</b>          | Spatial Pyramid Pooling — MaxPool nhiều kích thước $\rightarrow$ tăng receptive field. SPPF: phiên bản nhanh hơn (nối tiếp thay song song). |
| <b>Re-parameterization</b> | Kỹ thuật training multi-branch, inference single-branch. Gộp nhiều Conv thành 1 Conv duy nhất bằng phép cộng ma trận.                       |
| <b>Attention</b>           | Cơ chế cho model “tập trung” vào vùng quan trọng. Self-attention so sánh mọi vị trí với nhau (global receptive field).                      |
| <b>FlashAttention</b>      | Thuật toán tính attention tối ưu memory — giảm memory footprint đáng kể, cho phép dùng attention trên feature maps lớn.                     |

## Kỹ thuật Detection

| Thuật ngữ               | Giải thích                                                                                                                                |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Bounding Box</b>     | Hình chữ nhật bao quanh vật thể, xác định vị trí (x, y, width, height).                                                                   |
| <b>Anchor Box</b>       | Hộp mẫu (template) được định nghĩa trước. Model dự đoán offsets so với anchor thay vì tọa độ tuyệt đối. v2-v7 dùng anchor, v8+ bỏ anchor. |
| <b>Anchor-free</b>      | Không dùng anchor. Dự đoán trực tiếp center point + 4 distances (khoảng cách từ tâm đến 4 cạnh). Đơn giản hơn anchor-based.               |
| <b>NMS</b>              | Non-Maximum Suppression — thuật toán loại bỏ bounding boxes trùng lặp. Giữ box confidence cao nhất, bỏ các boxes có IoU lớn.              |
| <b>NMS-free</b>         | Không cần NMS — model tự output 1 box/vật. Từ v10 trở đi. Dùng dual assignment (o2m + o2o).                                               |
| <b>Decoupled Head</b>   | Tách classification và regression thành 2 branches riêng biệt. YOLOX giới thiệu, v6, v8+ adopt.                                           |
| <b>Label Assignment</b> | Chiến lược gán prediction cho ground truth khi training. SimOTA (động), TAL (task-aligned), STAL (small-target-aware).                    |
| <b>End-to-End</b>       | Pipeline từ input đến output không cần bước xử lý thủ công trung gian (ví dụ: không cần NMS).                                             |

| Thuật ngữ             | Giải thích                                                                                                   |
|-----------------------|--------------------------------------------------------------------------------------------------------------|
| Multi-scale           | Dự đoán ở nhiều kích thước (scales). YOLOv3+: 3 scales (lớn/trung/nhỏ) để detect vật ở mọi kích thước.       |
| One-stage / Two-stage | One-stage: 1 lần forward pass ra kết quả (YOLO). Two-stage: region proposal + classification (Faster R-CNN). |

## Metrics (Chỉ số đánh giá)

| Thuật ngữ      | Giải thích                                                                                                                                           |
|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| IoU            | Intersection over Union — đo mức trùng lặp giữa predicted box và ground truth. $IoU = \frac{Giao}{Hop}$ . Từ 0 (không trùng) đến 1 (trùng hoàn hảo). |
| mAP            | mean Average Precision — trung bình AP trên tất cả classes. Metric chính đánh giá detection.                                                         |
| mAP@50         | AP ở ngưỡng $IoU \geq 0.5$ . Dùng trên VOC.                                                                                                          |
| mAP@50:95      | Trung bình AP ở IoU từ 0.5 đến 0.95 (bước 0.05). Metric chính trên COCO — khắt khe hơn mAP@50.                                                       |
| FPS            | Frames Per Second — số ảnh xử lý mỗi giây. $\geq 30$ FPS = real-time.                                                                                |
| Params         | Số lượng tham số của model. Ví dụ: 3.2M = 3.2 triệu tham số. Ít params = model nhẹ.                                                                  |
| FLOPs / GFLOPs | Floating Point Operations — số phép tính cần thực hiện. Đo computational cost. 1 GFLOPs = 1 tỷ phép tính.                                            |
| Latency        | Thời gian xử lý 1 ảnh (ms). Khác FPS vì bao gồm cả pre/post-processing.                                                                              |

## Training (Huấn luyện)

| Thuật ngữ                  | Giải thích                                                                                                                           |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <b>Batch Normalization</b> | Chuẩn hóa output mỗi layer theo mini-batch $\rightarrow$ training ổn định hơn, nhanh hội tụ hơn. YOLOv2 thêm BN: +2% mAP.            |
| <b>Mosaic</b>              | Data augmentation ghép 4 ảnh thành 1 $\rightarrow$ model thấy nhiều context, giảm batch size cần thiết. YOLOv4 giới thiệu.           |
| <b>CutMix</b>              | Cắt 1 vùng từ ảnh A dán vào ảnh B, mix labels theo tỷ lệ diện tích.                                                                  |
| <b>Augmentation</b>        | Biến đổi ảnh khi training (xoay, lật, zoom, đổi màu...) để model tổng quát hơn.                                                      |
| <b>Transfer Learning</b>   | Dùng model đã train trên dataset lớn (ImageNet) $\rightarrow$ fine-tune trên dataset nhỏ của mình. Tiết kiệm thời gian, ít data hơn. |
| <b>Quantization</b>        | Giảm precision từ FP32 (32-bit) xuống INT8 (8-bit) $\rightarrow$ model nhẹ hơn 4 $\times$ , nhanh hơn, nhưng có thể mất accuracy.    |
| <b>PTQ</b>                 | Post-Training Quantization — quantize sau khi train xong. Nhanh nhưng mất accuracy.                                                  |
| <b>QAT</b>                 | Quantization-Aware Training — train lại với simulated quantization $\rightarrow$ giữ accuracy tốt hơn PTQ.                           |
| <b>BoF / BoS</b>           | Bag of Freebies (tăng training cost, không tăng inference) / Bag of Specials (tăng nhẹ inference cost). YOLOv4 hệ thống hóa.         |

## Loss Functions (Hàm mất mát)

| Thuật ngữ        | Giải thích                                                                                                                                  |
|------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| <b>MSE / SSE</b> | Mean/Sum Squared Error — đo sai số bình phương. v1–v3 dùng cho box coordinates.                                                             |
| <b>IoU Loss</b>  | $\mathcal{L} = 1 - IoU$ . Trực tiếp tối ưu IoU thay vì tọa độ riêng lẻ.                                                                     |
| <b>GIoU</b>      | Generalized IoU — xét cả vùng bao ngoài 2 boxes. Giải quyết trường hợp 2 box không giao nhau ( $IoU = 0$ nhưng $GIoU \neq 0$ ).             |
| <b>CIoU</b>      | Complete IoU — thêm khoảng cách tâm + aspect ratio vào GIoU. $\mathcal{L}_{CIoU} = 1 - IoU + \frac{\rho^2}{c^2} + \alpha v$ . YOLOv4+ dùng. |

| Thuật ngữ  | Giải thích                                                                                                                    |
|------------|-------------------------------------------------------------------------------------------------------------------------------|
| <b>DFL</b> | Distribution Focal Loss — dự đoán phân phối xác suất cho box coordinates (16 bins) thay vì 1 giá trị. v8–v12 dùng, YOLO26 bỏ. |
| <b>BCE</b> | Binary Cross-Entropy — loss cho classification. v3+ dùng BCE thay Softmax để hỗ trợ multi-label.                              |

## Deployment (Triển khai)

| Thuật ngữ              | Giải thích                                                                                                    |
|------------------------|---------------------------------------------------------------------------------------------------------------|
| <b>ONNX</b>            | Open Neural Network Exchange — format chuẩn để export model, chạy được trên nhiều frameworks/hardware.        |
| <b>TensorRT</b>        | NVIDIA SDK tối ưu inference trên GPU NVIDIA. Tăng 2–5× speed so với PyTorch.                                  |
| <b>Edge deployment</b> | Triển khai model trên thiết bị nhỏ (mobile, camera, drone) thay vì server lớn. Cần model nhẹ, ít computation. |
| <b>FP16 / FP32</b>     | Float 16-bit / 32-bit precision. FP16 nhanh hơn, ít memory hơn, gần như không mất accuracy.                   |
| <b>INT8</b>            | Integer 8-bit. Nhẹ hơn FP32 gấp 4× nhưng có thể mất 0.5–2% accuracy.                                          |